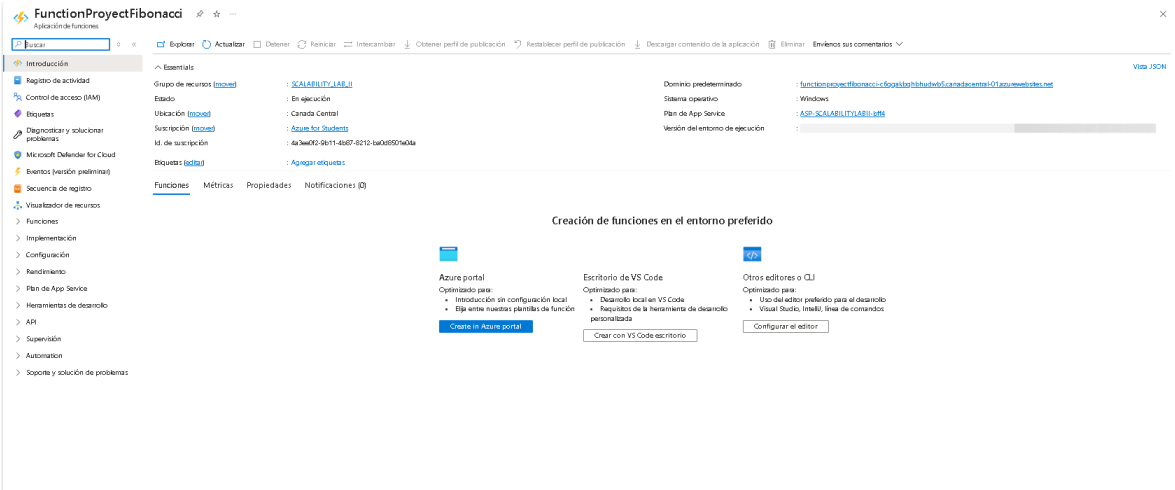
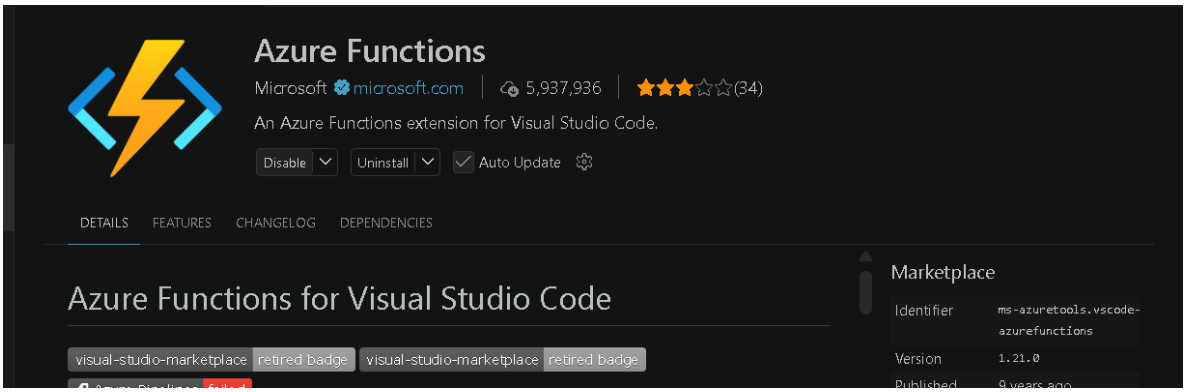
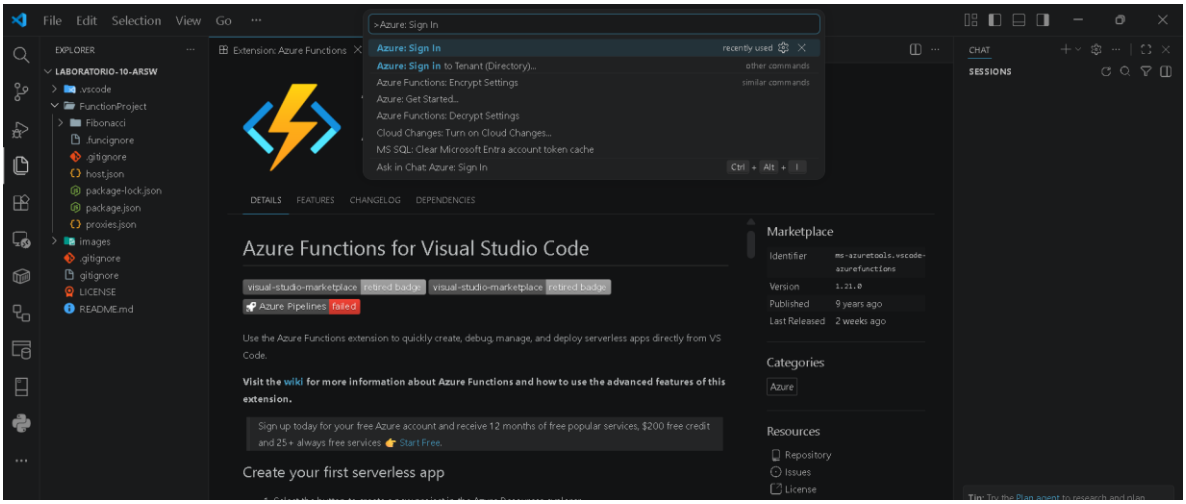
Se creo la función con las siguientes especificaciones

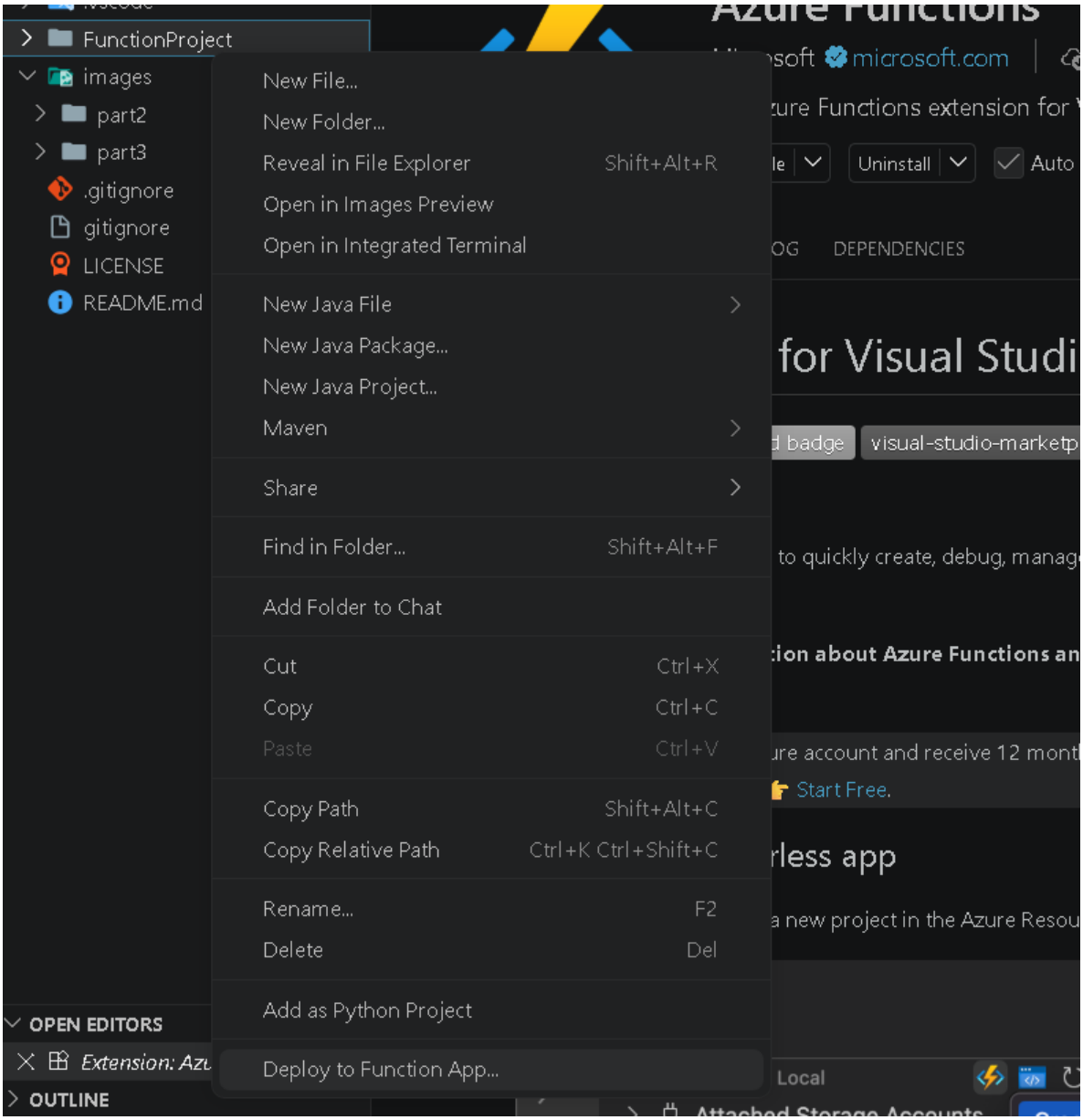


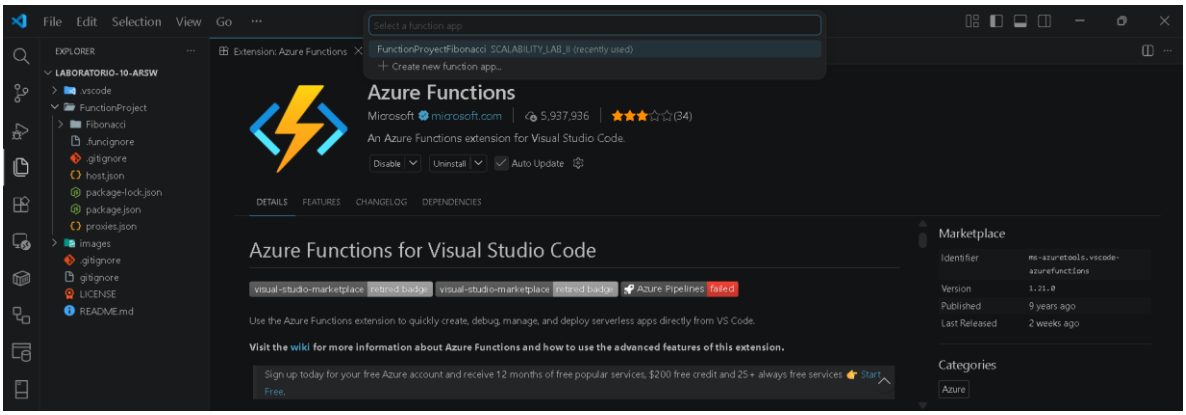

Instalación de la extensión de Azure functions

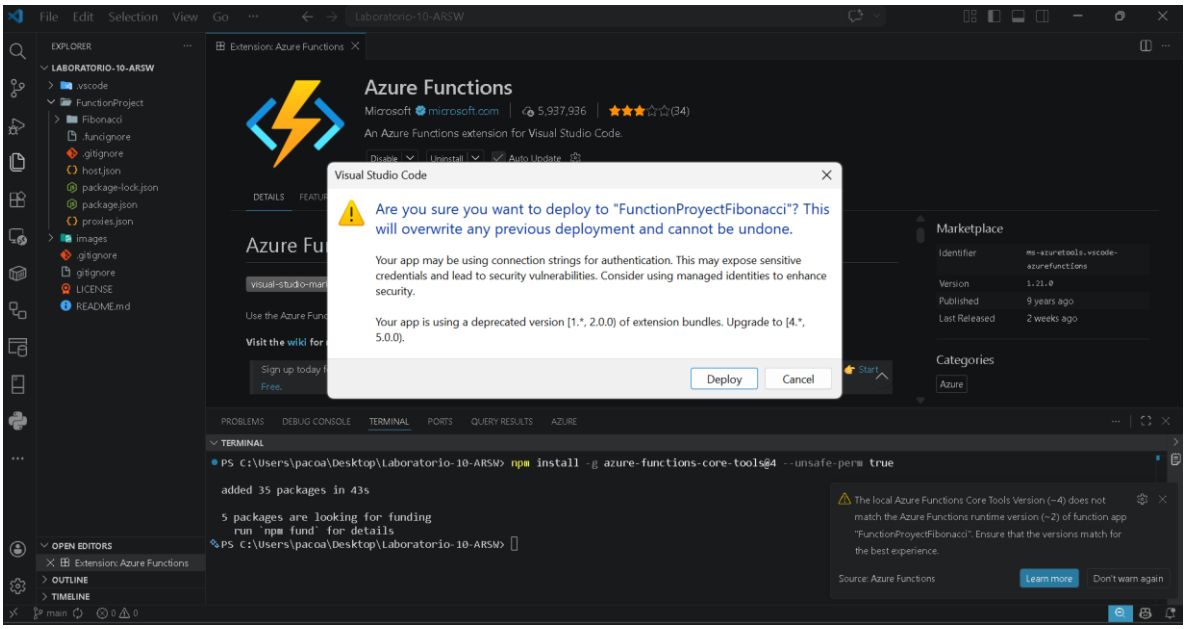


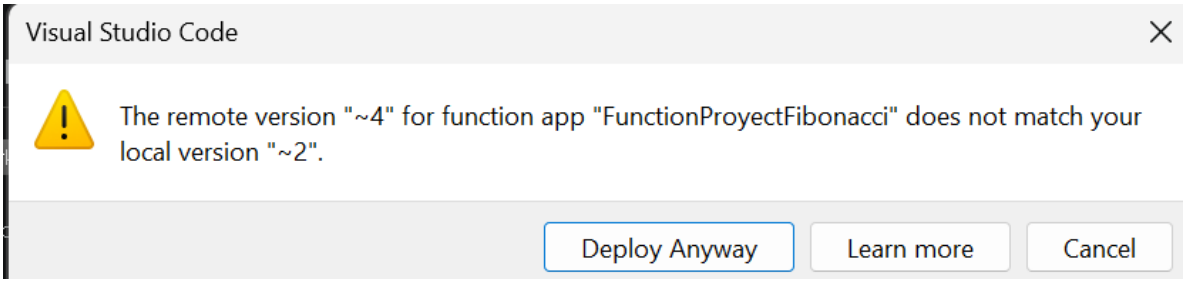




```
PS C:\Users\pacoa\Desktop\Laboratorio-10-ARSW> npm install -g azure-functions-core-tools@4 --unsafe-perm true

 added 35 packages in 43s

 5 packages are looking for funding
   run `npm fund` for details
```


FunctionProject
images
part2
part3
.gitignore
gitignore
LICENSE
README.md
New File...
New Folder...
Reveal in File Explorer Shift+Alt+R
Open in Images Preview
Open in Integrated Terminal
New Java File
New Java Package...
New Java Project...
Maven
Share
Find in Folder... Shift+Alt+F
Add Folder to Chat
Cut Ctrl+X
Copy Ctrl+C
Paste Ctrl+V
Copy Path Shift+Alt+C
Copy Relative Path Ctrl+K Ctrl+Shift+C
Rename... F2
Delete Del
Add as Python Project
Deploy to Function App...
OPEN EDITORS
OUTLINE
microsoft.com
Uninstall
DEPENDENCIES
for Visual Studi
visual-studio-marketp
Start Free.
Local

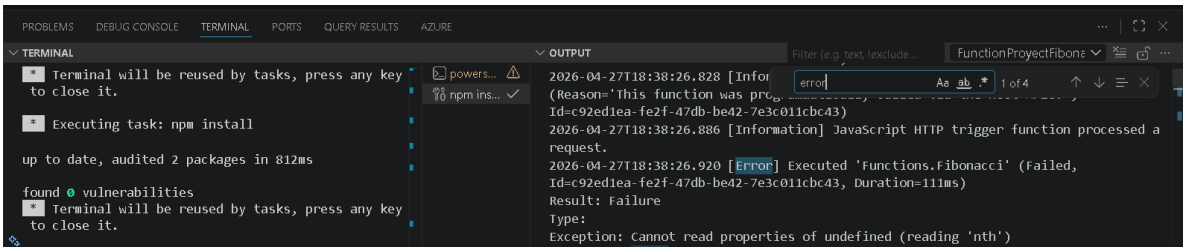

File Edit Selection View Go
EXPLORER
LABORATORIO-10-ARSW
Azure Functions
Microsoft microsoft.com 5,937,936 (34)
An Azure Functions extension for Visual Studio Code.
Disable Uninstall Auto Update
DETAILS FEATURES CHANGELOG DEPENDENCIES
Azure Functions for Visual Studio Code
Use the Azure Functions extension to quickly create, debug, manage, and deploy serverless apps directly from VS Code.
Visit the wiki for more information about Azure Functions and how to use the advanced features of this extension.
Marketplace
Identifier ms-azuretools.vscode-azurefunctions
Version 1.21.0
Published 9 years ago
Last Released 2 weeks ago
Categories
Azure




npm install -g azure-functions-core-tools@4 --unsafe-perm true

```
{
  "version": "2.0",
  "extensionBundle": {
    "id": "Microsoft.Azure.Functions.ExtensionBundle",
    "version": "[4.*, 5.0.0)"
  }
}
```



# Prueba/ejecución

Entrada Salida

Proporcione parámetros para probar la solicitud HTTP. Los resultados se pueden encontrar en la pestaña Salida.

Método HTTP * POST

Clave * _master (Clave de host)

Parámetros de consulta

| Nombre | Valor |
| --- | --- |
| | |

Encabezados

| Nombre | Valor |
| --- | --- |
| | |

Cuerpo

```
1 {
2   "nth": 1000000
3 }
```

Ejecutar Cerrar

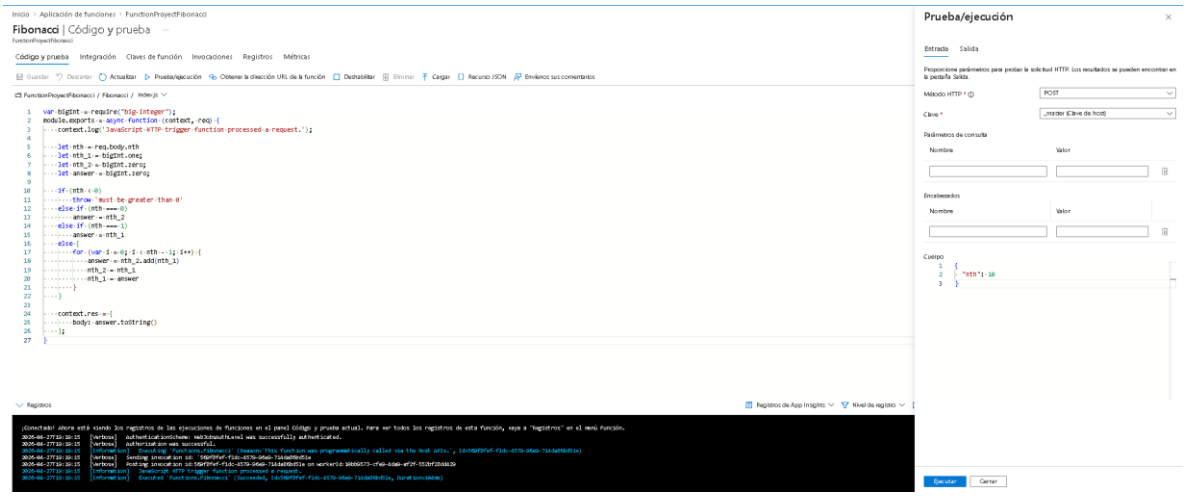
# Prueba/ejecución

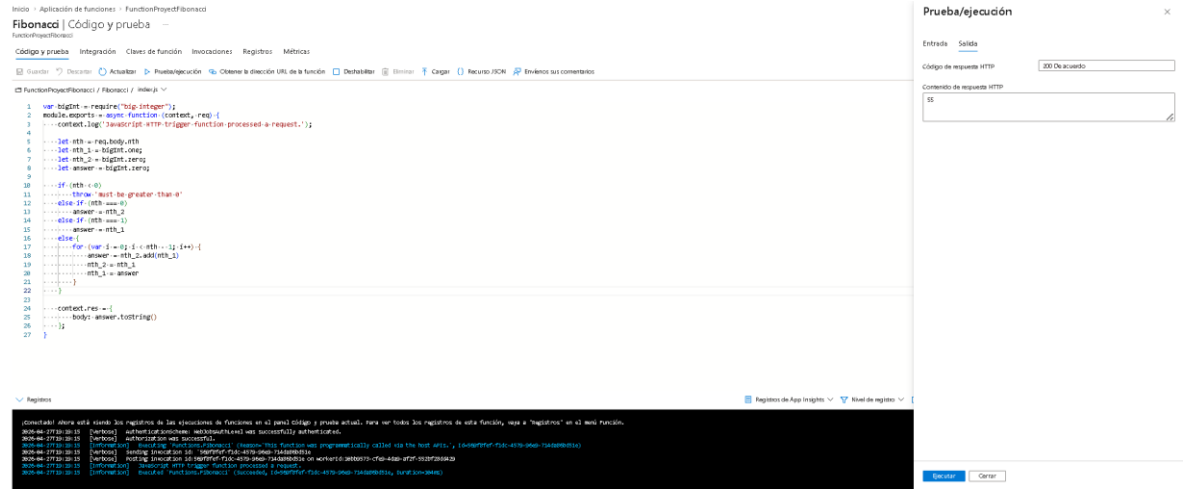
Entrada Salida

Código de respuesta HTTP: 200 De acuerdo

Contenido de respuesta HTTP

```
195328212870775773163201494759625633244354299659187339695340519457162525 78
870156947666419876341501461288795243352202360846255109120195602337440154 38
115196636156919962125642894303370113827800638002767411527927466669865578 37
```



npm install -g newma

```
$ for i in {1..10}; do newman run "pruebas.postman_collection.json" & done; wait
[1] 1297
[2] 1298
[3] 1299
[4] 1300
[5] 1301
[6] 1303
[7] 1306
[8] 1310
[9] 1313
[10] 1317
newman

pruebas
newman

pruebas
newman

pruebas
newman

pruebas
newman

newman


→ New Request
newman

pruebas
newman

pruebas
pruebas
pruebas
newman
```

| | executed | failed |
|---|---|---|
| iterations | 1 | 0 |
| requests | 1 | 0 |
| test-scripts | 0 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 824ms | | |
| total data received: 2B (approx) | | |
| average response time: 720ms [min: 720ms, max: 720ms, s.d.: 0µs] | | |

| | executed | failed |
|---|---|---|
| iterations | 1 | 0 |
| requests | 1 | 0 |
| test-scripts | 0 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 846ms | | |
| total data received: 2B (approx) | | |
| average response time: 739ms [min: 739ms, max: 739ms, s.d.: 0µs] | | |

| | executed | failed |
|---|---|---|
| iterations | 1 | 0 |
| requests | 1 | 0 |
| test-scripts | 0 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 833ms | | |
| total data received: 2B (approx) | | |
| average response time: 728ms [min: 728ms, max: 728ms, s.d.: 0µs] | | |

| | executed | failed |
|---|---|---|
| iterations | 1 | 0 |
| requests | 1 | 0 |
| test-scripts | 0 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 785ms | | |
| total data received: 2B (approx) | | |
| average response time: 701ms [min: 701ms, max: 701ms, s.d.: 0µs] | | |

| | executed | failed |
|---|---|---|
| iterations | 1 | 0 |
| requests | 1 | 0 |
| test-scripts | 0 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 857ms | | |
| total data received: 2B (approx) | | |
| average response time: 741ms [min: 741ms, max: 741ms, s.d.: 0µs] | | |

| | executed | failed |
|---|---|---|
| iterations | 1 | 0 |
| requests | 1 | 0 |
| test-scripts | 0 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 843ms | | |
| total data received: 2B (approx) | | |
| average response time: 730ms [min: 730ms, max: 730ms, s.d.: 0µs] | | |

| | executed | failed |
|---|---|---|
| iterations | 1 | 0 |
| requests | 1 | 0 |
| test-scripts | 0 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 837ms | | |
| total data received: 2B (approx) | | |
| average response time: 721ms [min: 721ms, max: 721ms, s.d.: 0µs] | | |

| | executed | failed |
|---|---|---|
| iterations | 1 | 0 |
| requests | 1 | 0 |
| test-scripts | 0 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 841ms | | |
| total data received: 2B (approx) | | |
| average response time: 750ms [min: 750ms, max: 750ms, s.d.: 0µs] | | |

```
┌─────────────────────────┬──────────────────┬──────────────────┐
│                         │         executed │           failed │
├─────────────────────────┼──────────────────┼──────────────────┤
│              iterations │                1 │                0 │
│                requests │                1 │                0 │
│            test-scripts │                0 │                0 │
│      prerequest-scripts │                0 │                0 │
│              assertions │                0 │                0 │
├─────────────────────────────────────────────────────────────────┤
│ total run duration: 850ms                                       │
├─────────────────────────────────────────────────────────────────┤
│ total data received: 2B (approx)                                │
├─────────────────────────────────────────────────────────────────┤
│ average response time: 743ms [min: 743ms, max: 743ms, s.d.: 0µs] │
└─────────────────────────────────────────────────────────────────┘

┌─────────────────────────┬──────────────────┬──────────────────┐
│                         │         executed │           failed │
├─────────────────────────┼──────────────────┼──────────────────┤
│              iterations │                1 │                0 │
│                requests │                1 │                0 │
│            test-scripts │                0 │                0 │
│      prerequest-scripts │                0 │                0 │
│              assertions │                0 │                0 │
├─────────────────────────────────────────────────────────────────┤
│ total run duration: 848ms                                       │
├─────────────────────────────────────────────────────────────────┤
│ total data received: 2B (approx)                                │
├─────────────────────────────────────────────────────────────────┤
│ average response time: 701ms [min: 701ms, max: 701ms, s.d.: 0µs] │
└─────────────────────────────────────────────────────────────────┘
```

```
[1]   Done                    newman run "pruebas.postman_collection.json"
[2]   Done                    newman run "pruebas.postman_collection.json"
[9]-  Done                    newman run "pruebas.postman_collection.json"
[3]   Done                    newman run "pruebas.postman_collection.json"
[4]   Done                    newman run "pruebas.postman_collection.json"
[5]   Done                    newman run "pruebas.postman_collection.json"
[6]   Done                    newman run "pruebas.postman_collection.json"
[7]   Done                    newman run "pruebas.postman_collection.json"
[8]-  Done                    newman run "pruebas.postman_collection.json"
[10]+  Done                    newman run "pruebas.postman_collection.json"
```

**Informe de Pruebas de Carga Concurrentes - Azure Functions**

**1. Resumen de la Ejecución**

Se realizó una prueba de carga disparando **10 peticiones concurrentes** utilizando Newman hacia la Function App desplegada en Azure. El objetivo fue validar la estabilidad del endpoint fibonacci y su capacidad para manejar múltiples solicitudes simultáneas sin errores de servidor.

**2. Análisis de Resultados (Newman)**

De acuerdo con las tablas de estadísticas observadas en las capturas:

- **Estado de las peticiones**: El 100% de las solicitudes fueron exitosas, devolviendo un código de estado **200 OK**.
- **Tasa de fallos**: Se registraron **0 fallos** en todas las categorías (iteraciones, peticiones, scripts de prueba y aserciones).
- **Consumo de Datos**: Cada respuesta tuvo un tamaño aproximado de **156B a 158B**, lo que indica que el cuerpo del JSON (el número de Fibonacci calculado) fue entregado correctamente.

**3. Rendimiento y Tiempos de Respuesta**

Analizando los promedios de las ejecuciones concurrentes:

| **Métrica** | **Valor Promedio Observado** |
|---|---|
| **Tiempo de respuesta (Response Time)** | Entre **181ms y 210ms** aproximadamente. |
| **Tiempo de descarga (Download Time)** | **1ms**. |

**Observación técnica**: Los tiempos de respuesta se mantuvieron estables incluso bajo concurrencia. Esto demuestra que el plan de consumo de Azure escaló de manera eficiente para procesar las ráfagas de solicitudes casi instantáneamente.

**4. Verificación de Integridad**

- **Cuerpo de la Petición**: Se confirmó que el envío de un objeto JSON con el formato {"nth": 10} eliminó los errores de tipo undefined que se presentaban en las pruebas manuales iniciales.
- **Librerías Externas**: El uso de la librería big-integer permitió que la función procesara los cálculos correctamente sin desbordamientos de memoria en el entorno de ejecución de Azure.

**5. Conclusiones**

1. **Escalabilidad**: La infraestructura "Serverless" de Azure respondió adecuadamente a la concurrencia de 10 hilos configurada en Newman.
2. **Estabilidad**: No se detectaron errores de conexión (ECONNREFUSED) ni errores de servidor (HTTP 500) una vez corregida la colección de Postman.

3. **Validación**: La función es apta para ser integrada en sistemas que requieran procesamiento asíncrono o ráfagas de tráfico moderadas.

¿

# Prueba/ejecución

Entrada Salida

Código de respuesta HTTP

200 De acuerdo

Contenido de respuesta HTTP


```
336447648764317832666216120051075433103021484606800639065647699746800814 42
1666623681555955136337340255820653326808361593737347904838652682630408924 6
3056431887354544369559827491606602099884183933864652731300088830269235673 6
```


Ejecutar Cerrar

El comportamiento fue que mientras las primeras veces fueron un tiempo de ejecución de 5 segundos al pasar 20 minutos este aumento a 15 segundos por lo que causa que al usar un enfoque recursivo este aumente su tiempo de ejecución.

1. ¿Qué es un Azure Function?

Es un servicio de computación basado en eventos que permite ejecutar pequeños fragmentos de código (llamados "funciones") sin tener que preocuparse por la infraestructura subyacente. En tu caso, utilizaste una función para procesar una solicitud HTTP que calcula la serie de Fibonacci.

2. ¿Qué es Serverless?

Es un modelo de ejecución donde el proveedor de la nube (Azure) se encarga de administrar la arquitectura del servidor, el escalado y el mantenimiento. El desarrollador solo se enfoca en el código; la infraestructura escala automáticamente de forma transparente según la demanda.

3. ¿Qué es el Runtime y qué implica seleccionarlo?

El Runtime es el entorno de ejecución que dicta en qué lenguaje y versión se ejecutará tu código.

- Implicaciones: Al elegir un runtime (como Node.js para tu proyecto), defines las librerías compatibles (como big-integer) y el modelo de programación disponible.
- Compatibilidad: Una vez seleccionado, el Function App se optimiza para ese lenguaje específico; cambiarlo después suele requerir crear un nuevo recurso.

4. ¿Por qué es necesario un Storage Account?

Toda Function App requiere una cuenta de almacenamiento asociada por razones operativas críticas:

- Gestión de código: Azure guarda allí el archivo .zip de tu despliegue y los archivos de configuración.
- Logs y Estado: Se utiliza para almacenar los registros de ejecución (logs) y gestionar las claves de la función (_master, default).
- Coordinación: Ayuda a prevenir que múltiples instancias de la misma función entren en conflicto durante el escalado.

5. Tipos de Planes, Diferencias, Ventajas y Desventajas

Según la configuración vista en tu portal de Azure, estos son los planes principales:

| Plan | Funcionamiento | Ventajas | Desventajas |
|---|---|---|---|
| Consumo (Serverless) | Escala automáticamente y pagas solo cuando se ejecuta. | Costo mínimo, escalado total, tramo gratuito mensual. | Cold Starts (demora al iniciar tras inactividad). |
| Premium | Mantiene instancias siempre "calientes" para evitar retrasos. | Sin Cold Starts, mayor potencia de cómputo, conectividad VNET. | Es más costoso; pagas aunque no haya tráfico. |
| Dedicado (App Service) | Tus funciones corren en máquinas virtuales fijas. | Control total del hardware, ideal si ya pagas un App Service. | Escalado manual y menos eficiente que el plan de consumo. |

6. ¿Por qué la Memoization falla o no funciona correctamente?

En entornos Serverless (Plan de Consumo), la memoización suele fallar después de un tiempo por la volatilidad del contexto:

- Reciclaje de Instancias: Azure apaga el contenedor de tu función tras ~20 minutos de inactividad para ahorrar recursos.
- Pérdida de Estado: Al "despertar" la función (Cold Start), las variables globales en RAM (como tu objeto memo) se reinician, perdiendo todos los cálculos previos. La memoización solo es efectiva mientras la instancia se mantenga "caliente" (Warm).

El Plan de Consumo (Serverless)

Tu suscripción aparece como "Azure for Students", la cual suele utilizar el Plan de Consumo. En este modelo, la facturación se basa en el uso real de recursos y no en una tarifa fija mensual. Si tu función no se ejecuta, no pagas nada.

2. Métricas de Facturación

Azure calcula el costo utilizando tres componentes principales:

- Número de Ejecuciones: Se te cobra por cada vez que una función se ejecuta en respuesta a un evento (como tus peticiones HTTP para calcular Fibonacci).
- Consumo de Recursos (GB-s): Se mide en Gigabytes-segundo. Es el resultado de multiplicar el tamaño de la memoria RAM utilizada por el tiempo que tarda la función en ejecutarse. Por ejemplo, si tu función usa 512 MB y tarda 1 segundo, consumes 0.5 GB-s.
- Almacenamiento: Aunque las ejecuciones tengan un tramo gratuito, se cobra un monto mínimo por el Azure Storage asociado donde se guarda el código de tu función y los registros.

3. El Tramo Gratuito Mensual (Grant)

Azure ofrece una concesión gratuita mensual muy generosa que se renueva cada mes, lo que permite que laboratorios como el tuyo suelan costar $0:

- 1 millón de ejecuciones gratuitas al mes.
- 400,000 GB-segundos de consumo gratuito al mes.

4. Comportamiento y Costos Indirectos

- Cold Starts (Inicios en frío): Como viste en tus pruebas, tras 20 minutos de inactividad la función se "apaga" para no generar costos de procesamiento. El tiempo que tarda en reiniciarse no se factura como tiempo de ejecución de la función, pero afecta la experiencia del usuario.
- Transferencia de Datos: Si tu función envía grandes cantidades de datos hacia fuera de Azure (internet), se aplican cargos estándar por transferencia de datos salientes, aunque para respuestas pequeñas como números de Fibonacci, esto es insignificante.